The Gradle Odyssey

A journey through reusing Gradle build configuration





Marthijs Berfelo

Software Engineer — your helmsman for this voyage

Charting safer courses through multi-module Gradle builds

github.com/Marthijs-Berfelo

linkedin.com/in/marthijs-berfelo

The Journey



Ithaca

Setting Sail

The Cyclops's Cave

The Sirens

Ithaca, Regained



Ithaca

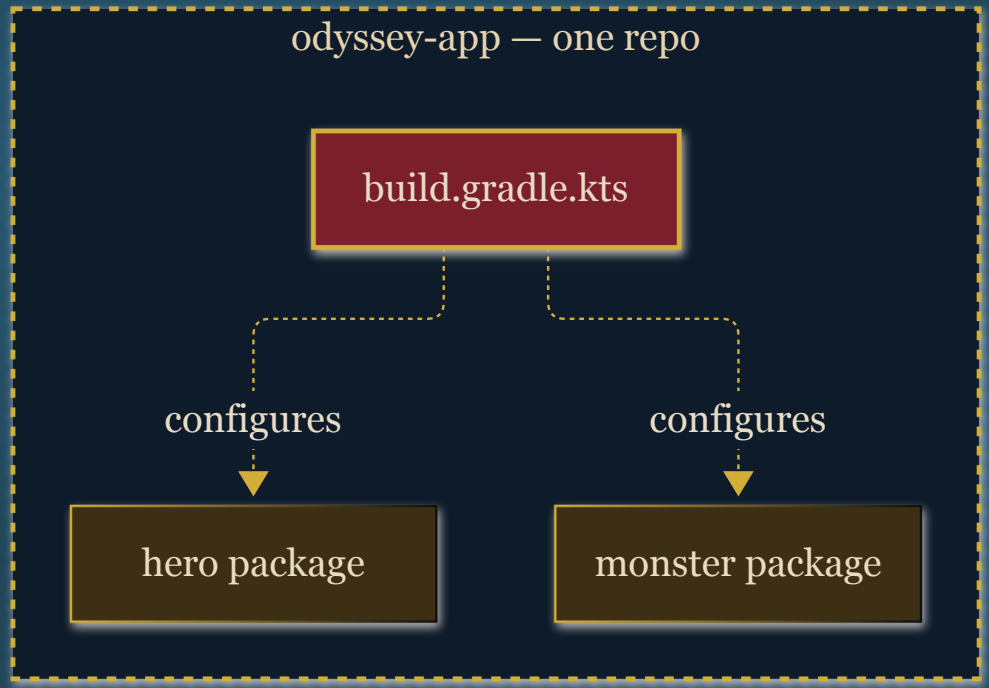
Single-module build — home base, where every project starts

One module, `odyssey-app`, holds both of our domain entities:

- **Hero** — `id`, `name`, `epithet`, `strength`
- **Monster** — `id`, `name`, `domain`, `danger`

One build, one deployable.

Schematic View



Code Demo

Pros & Cons

Pros

- Zero setup — one `build.gradle.kts`, one command to build and run
- Single dependency graph, trivial to reason about
- No build-logic duplication, because there's only one build

Cons

- Heroes and Monsters can't deploy independently
- A change to either domain forces rebuilding and redeploying both
- Team ownership boundaries blur inside one module

Conclusion

One module, one build — but Heroes and Monsters need to ship on separate schedules.

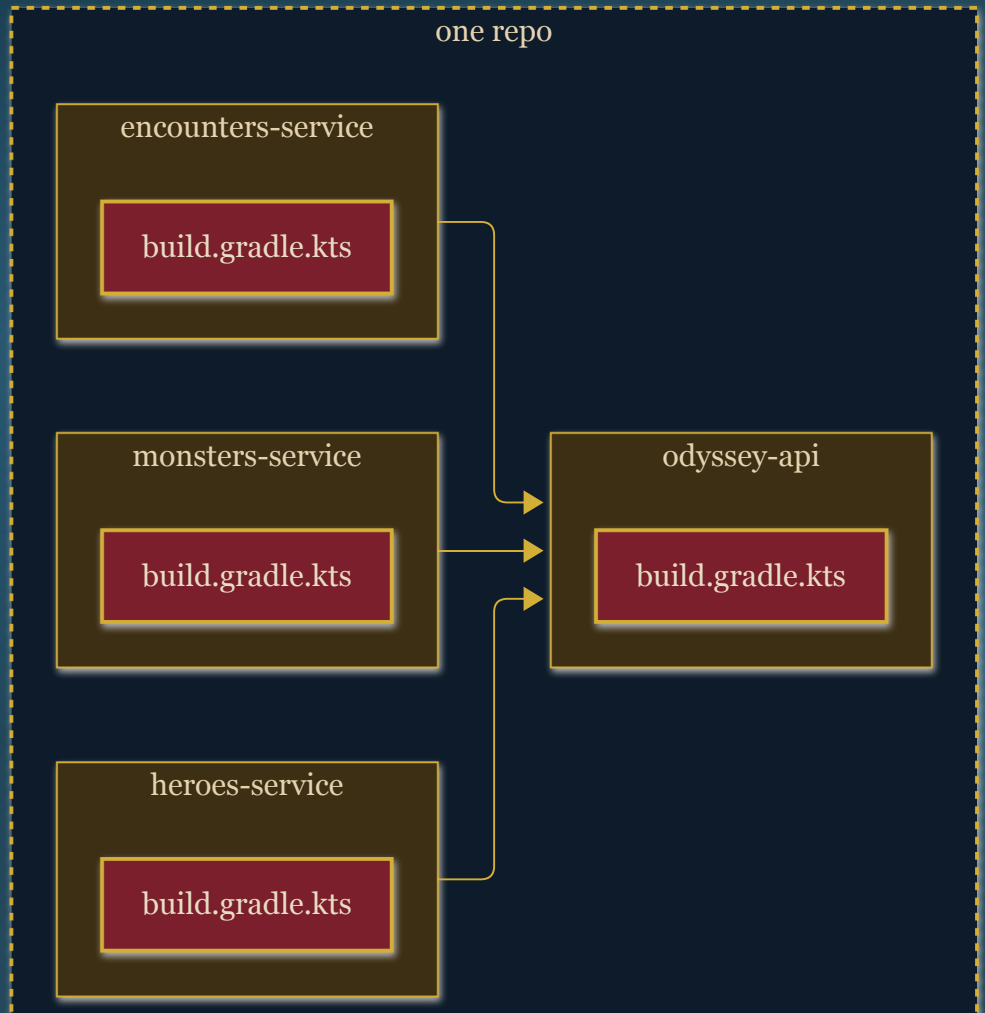


Setting Sail

Multi-module build — a fleet of modules, one voyage

Hero, Monster, and Encounter set sail as their own services, sharing one `api-spec`.

Schematic View



Code Demo

Pros & Cons

Pros

- `heroes-service`, `monsters-service`, and `encounters-service` deploy independently
- Clear module boundaries mirror service boundaries
- `odyssey-api` gives every service a single shared contract

Cons

- Spring Boot, Kotlin, and OpenAPI-codegen setup copy-pasted across 4 modules
- Version bumps must be repeated in every module's `build.gradle.kts`
- No shared place to fix a build mistake — it must be fixed four times

Conclusion

Independent deployability, at the cost of the same build setup — copy-pasted four times over.

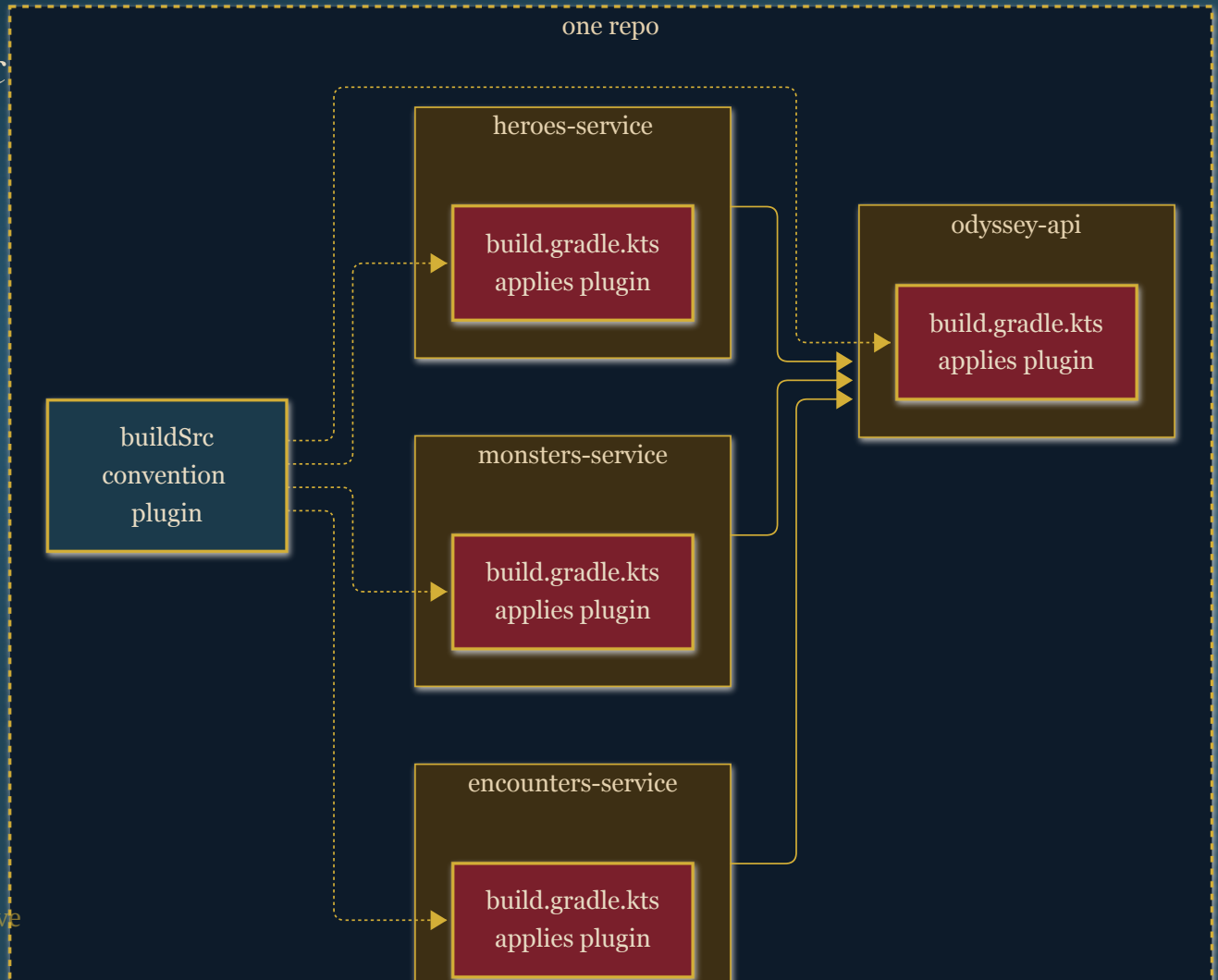


The Cyclops's Cave

`buildSrc` — powerful, but trapped on one island

One `buildSrc` convention plugin replaces four copies of build boilerplate.

Schematic


one repo
heroes-service
build.gradle.kts
applies plugin
odyssey-api
build.gradle.kts
applies plugin
buildSrc
convention
plugin
monsters-service
build.gradle.kts
applies plugin
encounters-service
build.gradle.kts
applies plugin

Code Demo

Pros & Cons

Pros

- Build logic deduplicated into `buildSrc` convention plugins
- One place to fix or evolve the Spring Boot/Kotlin conventions
- Modules apply a single plugin id instead of hand-rolled blocks

Cons

- `buildSrc` is trapped inside this repo — no other project can reuse it
- Reusing it elsewhere means copy-pasting `buildSrc` itself
- Any change inside `buildSrc` invalidates the whole build's configuration cache

Conclusion

`buildSrc` is a cave, not a harbor — nothing inside it can leave this repo.

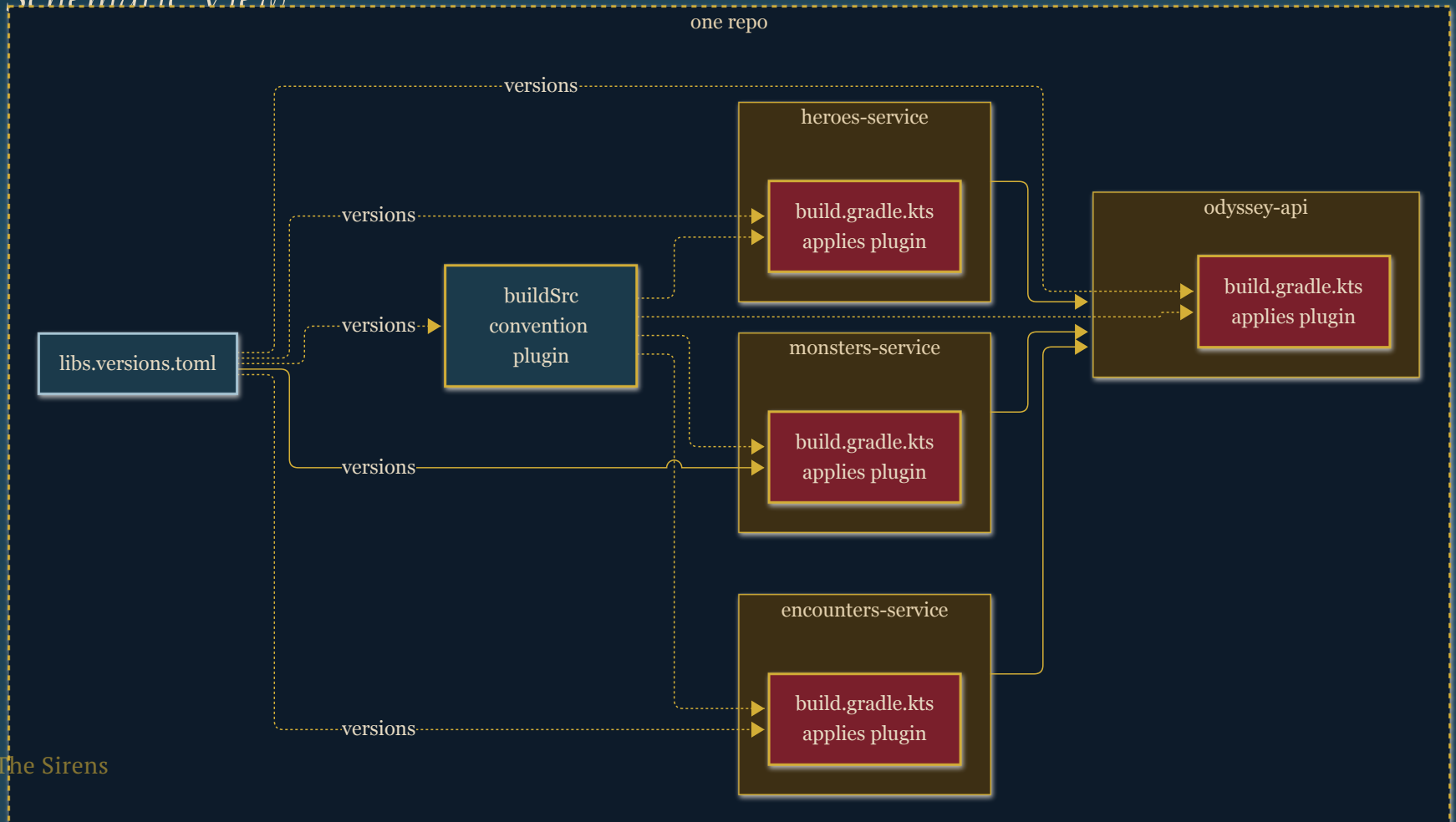


The Sirens

Version catalogs & BOMs — the tempting shortcut of centralizing versions

One `libs.versions.toml` aligns every dependency version, everywhere.

Schematic View



Code Demo

Pros & Cons

Pros

- One file (`libs.versions.toml`) aligns every dependency version
- No more version drift between modules and `buildSrc`
- Upgrading a library is a one-line change

Cons

- Doesn't solve cross-repo reuse — only versions are centralized, not build logic
- The catalog file itself must still be copy-pasted into any other repo
- Catalog syntax adds a small learning curve versus plain version strings

Conclusion

Versions are aligned everywhere — but the build logic is still steering straight for the cliffs.



Ithaca, Regained

Published, shared plugin — home again, transformed

`buildSrc` becomes a published plugin — reusable far beyond this repo.

Schematic View



Code Demo

Pros & Cons

Pros

- Build logic published once, consumed by any project via `plugins { id(...) }`
- True reuse — no copy-pasting `buildSrc` or the catalog into new repos
- Central place to version and release build-logic changes

Cons

- Adds publishing and versioning ceremony for the plugin itself
- Requires a real (or local) plugin repository and release discipline
- Shared build-logic changes now need their own release before consumers pick them up

Conclusion

Home again — the squatters driven out, one true build logic in their place.

The Journey, Complete

Five stages, one build — from a single module to a plugin the world can reuse.

Thank You



github.com/Marthijs-Berfelo/the-gradle-odyssey